

Usando *regex*

Expressões regulares descrevem padrões de texto: "quatro dígitos", "uma palavra depois de #", "uma data no formato DD/MM/AAAA". No Python, elas ficam no módulo `re`. Nesta aula: buscar, testar e extrair com `search`, `match`, `fullmatch`, `findall` e `finditer`, capturar partes com grupos, substituir com `sub`, dividir com `split` e por que usar strings cruas (`r"..."`).

`re.search` · `match` · `fullmatch`

Metacaracteres

`findall` · `finditer`

Grupos nomeados

`re.sub` · `split`

`r"strings cruas"`

Formação Claude Code 2026 · Baseado em docs.python.org (re – Regular expression operations; Regular Expression HOWTO), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

search, match e fullmatch

buscar.py

```
import re

texto = "Pedido 4821 enviado em 05/10/2026"

print(re.search(r"\d+", texto))      # primeira ocorrência
print(re.search(r"\d+", texto).group())
print(re.match(r"\d+", texto))       # só no começo: None
print(re.fullmatch(r"\d{4}", "4821")) # a string inteira
print(bool(re.fullmatch(r"\d{4}", "48210")))
```

SAÍDA

```
<re.Match object; span=(7, 11), match='4821'>
4821
None
<re.Match object; span=(0, 4), match='4821'>
False
```

FUNÇÃO

PROCURA

`re.search`

a primeira ocorrência, em qualquer lugar

`re.match`

só no começo da string

`re.fullmatch`

a string inteira tem de casar

✓ Se acha, devolve um objeto `Match`; senão, `None`

✓ `.group()` pega o texto encontrado; `span` diz onde estava

Os metacaracteres *mais usados*

SÍMBOLO	SIGNIFICA
<code>\d</code> · <code>\D</code>	dígito · não dígito
<code>\w</code> · <code>\W</code>	letra, número ou <code>_</code> · o contrário
<code>\s</code> · <code>\S</code>	espaço em branco · o contrário
<code>.</code>	qualquer caractere (menos quebra de linha)
<code>[abc]</code> · <code>[^abc]</code>	um dos listados · nenhum deles
<code>^</code> · <code>\$</code>	começo · fim do texto
<code>\b</code>	borda de palavra

QUANTIFICADOR	QUANTAS VEZES
<code>*</code>	zero ou mais
<code>+</code>	uma ou mais
<code>?</code>	zero ou uma (opcional)
<code>{4}</code> · <code>{4,5}</code>	exatamente 4 · de 4 a 5
<code>()</code>	grupo: captura uma parte
<code>a b</code>	a ou b
<code>\.</code> · <code>\(</code>	o próprio ponto · parêntese

Para usar um metacaractere como texto comum, escape com `\` :
`\.` casa só com ponto.

findall: *todas as ocorrências*

todos.py

```
import re

post = "Lançou! #python #regex custa R$ 49,90 ou 3x de 18,00"

print(re.findall(r"#\w+", post))           # hashtags
print(re.findall(r"\d+,\d{2}", post))      # preços
print(re.findall(r"\b\d+x\b", post))      # parcelas
```

SAÍDA

```
['#python', '#regex']
['49,90', '18,00']
['3x']
```

- ✓ `findall` devolve uma **lista** com cada texto encontrado
- ✓ `#\w+` : o caractere `#` seguido de uma ou mais letras
- ✓ `\d+,\d{2}` : dígitos, vírgula e exatamente dois dígitos
- ✓ `\b` evita pegar o "3x" no meio de uma palavra maior

Capturando *as partes*

grupos.py

```
import re

agenda = "Prova 12/11/2026, entrega 03/12/2026"
padrao = r"(?P<dia>\d{2})/(?P<mes>\d{2})/(?P<ano>\d{4})"

for m in re.finditer(padrao, agenda):
    print(m.group(), "->", m["dia"], m["mes"], m["ano"])

primeira = re.search(padrao, agenda)
print(primeira.groupdict())
print(re.findall(padrao, agenda))  # com grupos: tuplas
```

SAÍDA

```
12/11/2026 -> 12 11 2026
03/12/2026 -> 03 12 2026
{'dia': '12', 'mes': '11', 'ano': '2026'}
[('12', '11', '2026'), ('03', '12', '2026')]
```

- ✓ `(?P<nome>...)` cria um grupo com nome: `m["dia"]`
- ✓ `finditer` é um iterador de objetos `Match` (aulas 01 e 02)
- ✓ `groupdict()` devolve os grupos nomeados como dicionário
- ✓ Com grupos, `findall` devolve tuplas com as partes

re.sub: *trocar pelo padrão*

substituir.py

```
import re

ficha = "CPF 123.456.789-09   tel   (11) 98765-4321"

print(re.sub(r"\s+", " ", ficha))           # espaços extras
print(re.sub(r"\d{3}\.\d{3}\.\d{3}-\d{2}", "***", ficha))
print(re.sub(r"\d", "#", "senha 2026"))

# Grupo na troca: inverte data de AAAA-MM-DD para DD/MM/AAAA
print(re.sub(r"(\d{4})-(\d{2})-(\d{2})", r"\3/\2/\1",
            "2026-10-05"))
```

SAÍDA

```
CPF 123.456.789-09 tel (11) 98765-4321
CPF ***   tel   (11) 98765-4321
senha #####
05/10/2026
```

- ✓ `re.sub(padrão, troca, texto)` troca todas as ocorrências
- ✓ `\s+` por `" "` junta espaços repetidos
- ✓ Na troca, `\1`, `\2` ... repetem os grupos capturados: dá para reordenar partes

O CPF e o telefone são números de exemplo.

Ajustes *do dia a dia*

flags_compile.py

```
import re

nomes = ["Ana", "ana", "ANA", "Mariana"]
ana = re.compile(r"^ana$", re.IGNORECASE) # compila uma vez
print([n for n in nomes if ana.search(n)])

print(re.split(r"\s*[,;]\s*", "maçã; uva,kiwi , pera"))
```

```
['Ana', 'ana', 'ANA']
['maçã', 'uva', 'kiwi', 'pera']
```

crua.py

```
print(len("\b"), len(r"\b")) # \b vira 1 caractere sem o r
print(r"C:\novo\texto")     # o r guarda a barra como está
```

```
1 2
C:\novo\texto
```

- ✓ `re.IGNORECASE` ignora maiúsculas e minúsculas
- ✓ `re.compile` prepara o padrão uma vez para usar muitas
- ✓ `re.split` divide por um padrão, não por um texto fixo
- ✓ Sem o `r`, `"\b"` vira o caractere de "voltar" e o padrão quebra; com `r"\b"`, a barra chega ao `re`

Regra prática: escreva **sempre** os padrões como `r"..."`.

Padronizar *telefones*

Enunciado: extraia os telefones de um texto bagunçado (com ou sem parênteses e espaços) e mostre todos no formato `(DD) NNNNN-NNNN`.

Uma resposta possível

```
import re

contatos = """Ana: 11 98765-4321
Bia (21)3456-7890 | Caio: (31) 99999-0000
Dani: 1234"""

padrao = r"\((?\d{2})\)?\s*(\d{4,5})-(\d{4})"
for ddd, parte1, parte2 in re.findall(padrao, contatos):
    print(f"{ddd} {parte1}-{parte2}")
```

SAÍDA

```
(11) 98765-4321
(21) 3456-7890
(31) 99999-0000
```

- ✓ `\(?:` e `\)?` : parênteses opcionais
- ✓ `\d{4,5}` aceita fixo e celular
- ✓ "1234" não tem o formato e fica de fora

Teste o que *ficou*

1. `re.match(r"\d", "a1")` devolve:

A) "1"

B) None

C) ["1"]

D) Erro

B. match só olha o começo.

2. `re.findall(r"\d+", "a1b22c333")` vale:

A) ["1", "2", "3"]

B) ["1", "22", "333"]

C) "122333"

D) [1, 22, 333]

B.

3. Qual padrão casa com "2026" inteiro e nada mais?

A) `re.search(r"\d")`

B) `re.fullmatch(r"\d{4}")`

C) `re.match(r"\d")`

D) `re.findall(r"\D")`

B.

4. Por que escrever `r"\d"` e não `"\d"`?

A) É mais rápido

B) Para a barra chegar intacta ao re

C) Para ignorar maiúsculas

D) É obrigatório no Python 3

B.

Padrões no lugar *de if e split*

`search` acha a primeira ocorrência, `match` olha o começo e `fullmatch` exige a string inteira. `findall` e `finditer` extraem tudo, grupos capturam as partes, `sub` troca e `split` divide. Escreva os padrões como `r"..."` e comece simples: regex boa é a que outra pessoa consegue ler.

`search` · `match` · `fullmatch`

`findall` · `finditer`

`(?P<nome>)`

`sub` · `split`

`r"..."`

Próxima aula → 07 · Validação de Dados

Fontes: docs.python.org/3/library/re.html; docs.python.org/3/howto/regex.html, consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.